Αναφορά 4ης Εργασίας: FDTD 2D solver σε CUDA

Ονοματεπώνυμο: Μιχαήλ Μπαλτζάκης | ΑΜ: 10989

GitHub: https://github.com/MixalisBaltzakis/exercise4.git

1. Εισαγωγή & Υλοποίηση

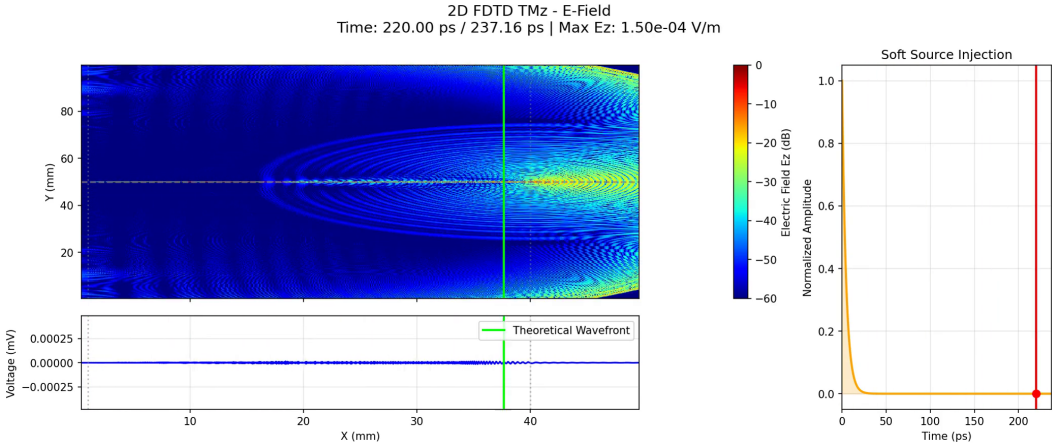
Στα πλαίσια της εργασίας, υλοποιήθηκε ένας 2D επιλυτής (solver) ηλεκτρομαγνητικών πεδίων βασισμένος στη μέθοδο Πεπερασμένων Διαφορών στο Πεδίο του Χρόνου (FDTD). Ο επιλυτής γράφτηκε εξ ολοκλήρου σε C++ / CUDA για εκτέλεση σε Κάρτα Γραφικών (GPU), αξιοποιώντας το πλέγμα του Yee. Ως συνοριακές συνθήκες απορρόφησης υλοποιήθηκαν Perfectly Matched Layers (PML) περιμετρικά του grid, εξασφαλίζοντας ότι τα κύματα δεν ανακλώνται πίσω στο πεδίο δοκιμών.

Επιπλέον, προσθέσαμε Soft Source διέγερσης με παραμετροποιήσιμες διηλεκτρικές σταθερές (Er), ώστε να μοντελοποιηθούν ρεαλιστικές διατάξεις όπως ανοιχτά τυπωμένα κυκλώματα (Microstrip) και σχισμές γείωσης (Ground Slits).

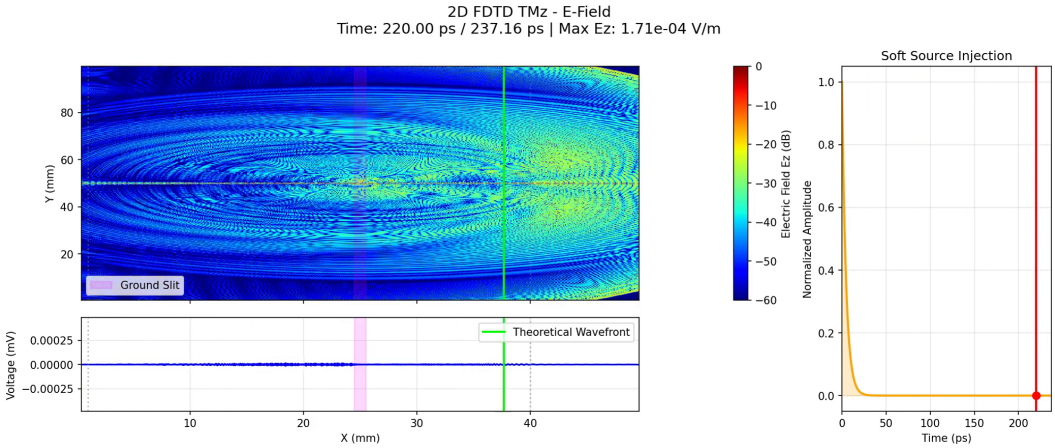
2. Αξιολόγηση Εφαρμογής (Πλεονεκτήματα - Αδυναμίες)

Ως εφαρμογή, ο 2D solver αποδείχτηκε εξαιρετικά ισχυρός στο να απεικονίσει φαινόμενα διάδοσης και ανακλάσεων. Στην περίπτωση του Microstrip (πρώτη εικόνα παρακάτω), διακρίνονται καθαρά οι ανακλάσεις λόγω ανοικτού κυκλώματος στο δεξί άκρο της γραμμής. Αντίστοιχα, στο Ground Slit (δεύτερη εικόνα) αποτυπώνεται όμορφα η ακτινοβολία της σχισμής σαν κεραία.

Ωστόσο, μια θεμελιώδης αδυναμία του 2D solver έναντι ενός πραγματικού 3D PCB, είναι ο τρόπος που αποσβένεται το κύμα. Στις δύο διαστάσεις (2D cylindrical wave), το κύμα υπακούει σε απόσβεση της τάξης του 1/sqrt(r), με αποτέλεσμα να διατηρεί το πλάτος του για πολύ μεγαλύτερες αποστάσεις απ' ό,τι στον 3D χώρο, όπου η απόσβεση είναι 1/r (spherical wave). Αυτό κάνει τα 2D μοντέλα λιγότερο ρεαλιστικά όσον αφορά την πλευρική διαρροή και τις παρεμβολές.



Εικόνα 1: Ανακλάσεις σε ανοικτό κύκλωμα Microstrip.



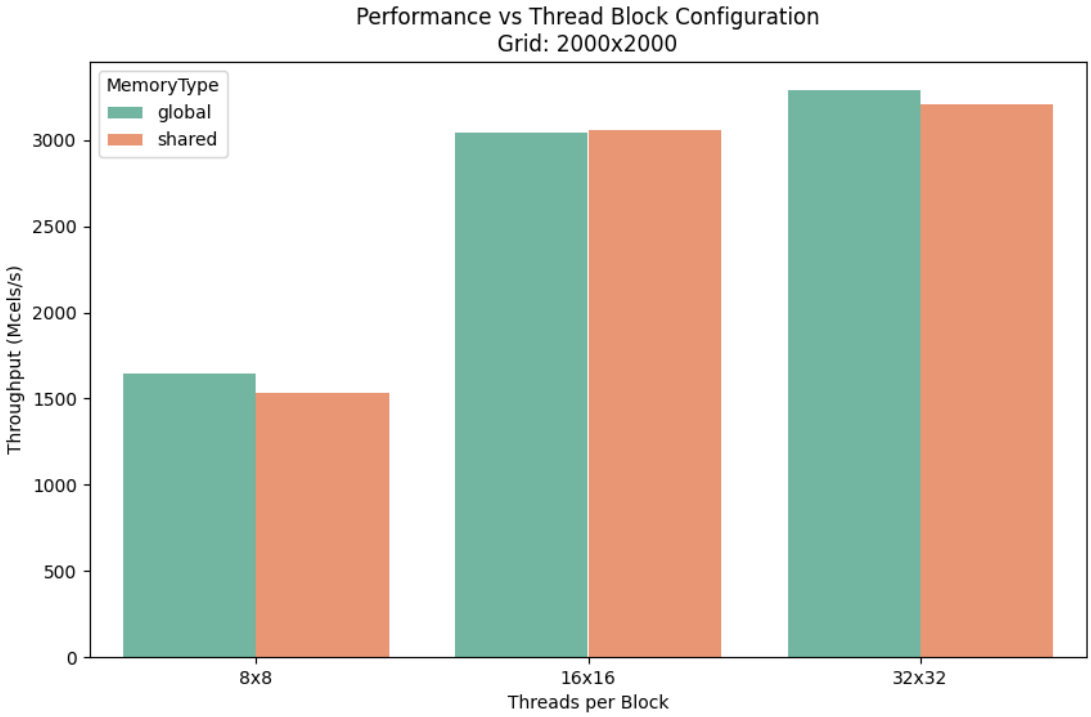
Εικόνα 2: Ακτινοβολία Σχισμής (Ground Slit).

3. Αρχιτεκτονική CUDA & Βελτιστοποίηση

Το κύριο αντικείμενο της εργασίας ήταν η μελέτη και βελτιστοποίηση του CUDA κώδικα. Ο κώδικας σχεδιάστηκε με δύο διαφορετικές στρατηγικές προσπέλασης μνήμης:
α) Απλή Global Memory, όπου κάθε thread διαβάζει τα γειτονικά κελιά απευθείας από τη VRAM.
β) Shared Memory Tiling με Halo Exchange, όπου τα blocks 16x16 (ή 32x32) φορτώνουν πρώτα τα δεδομένα στην τοπική Shared μνήμη του SM, με χρήση __syncthreads() για αποφυγή race conditions.

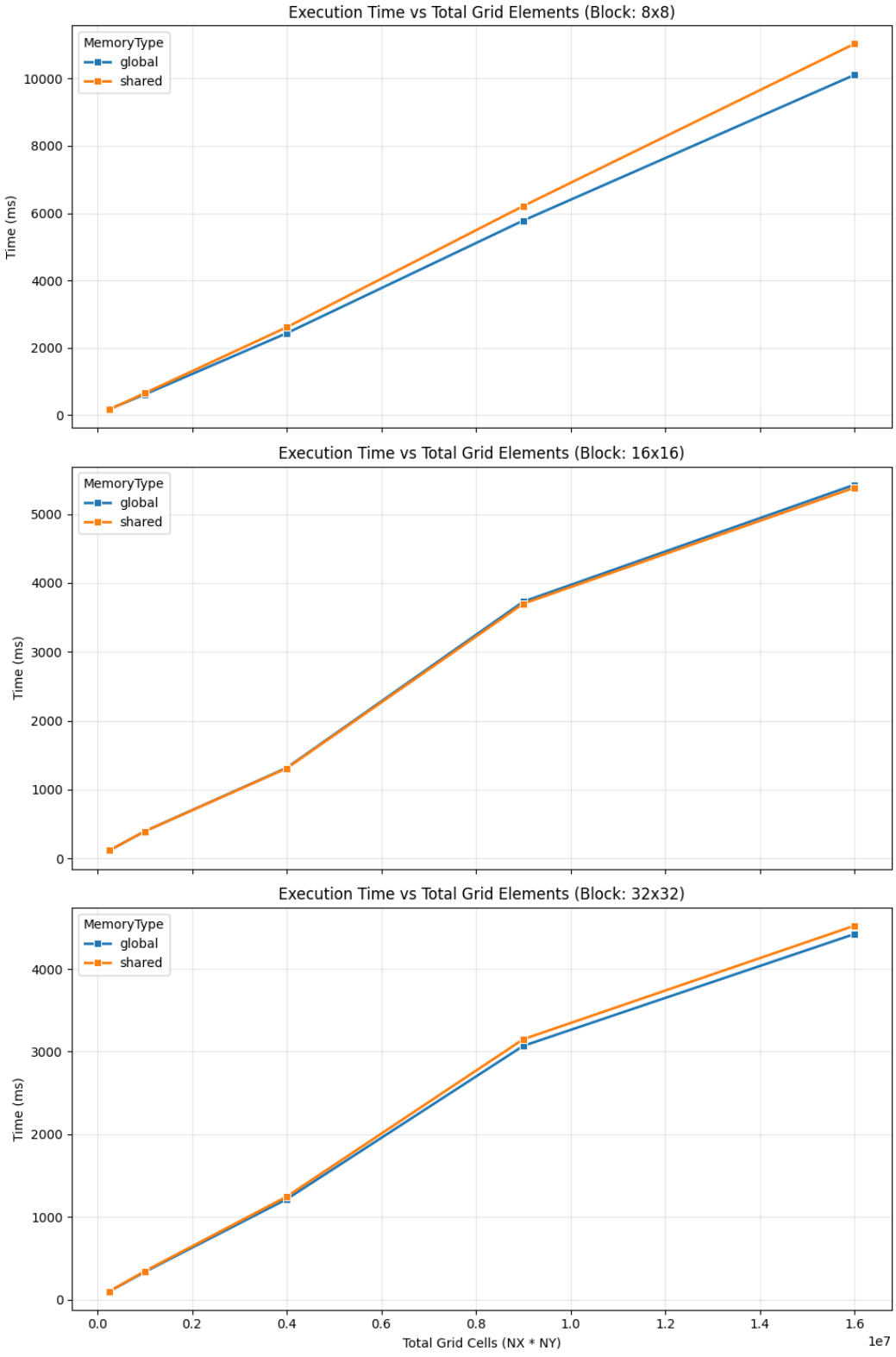
Τρέξαμε εξαντλητικά benchmarks στην GPU του cluster, εξάγοντας τη μέση τιμή (average) από 5 ανεξάρτητα runs των 1000 time steps το καθένα για μέγιστη στατιστική αξιοπιστία. Αναλύοντας τα Mcels/s (εκατομμύρια κελιά ανά δευτερόλεπτο), τα αποτελέσματα ήταν άκρως ενδιαφέροντα.

3.1 Ανάλυση Benchmarks



Όπως φαίνεται στο παραπάνω γράφημα, η Shared Memory δεν πρόσφερε κάποια δραματική επιτάχυνση έναντι της Global. Μάλιστα, η Global τα πήγε οριακά καλύτερα στις περισσότερες περιπτώσεις. Αυτό αποδεικνύει την τεράστια ισχύ της L1 Cache των σύγχρονων GPUs! Η L1 Cache αναλαμβάνει να κάνει αυτόματα αυτό που εμείς προγραμματίσαμε χειροκίνητα στη Shared Memory, γλιτώνοντας παράλληλα τον 'νεκρό' χρόνο του συγχρονισμού (__syncthreads()).

3.2 Συγκριτική Ανάλυση Χρόνου & Επιλογή Sweet Spot



Αναλύοντας τον καθαρό χρόνο εκτέλεσης (πάνω γράφημα) παρατηρούμε την τέλεια γραμμική κλιμάκωση (O(N)). Συγκρίνοντας όμως κάθετα τα 3 subplots, συμπεραίνουμε πως η μέγιστη απόδοση της κάρτας γραφικών επιτυγχάνεται στο block configuration 32x32.
Το 32x32 αποτελεί το Sweet Spot (1024 threads, μέγιστο hardware όριο), προσφέροντας το βέλτιστο occupancy στους Streaming Multiprocessors.

Συνεπώς, στον τελικό παραδοτέο κώδικα (video_pml.cu), προσαρμόσαμε τα dimensions σε dim3 threadsPerBlock(32, 32) και τους static __shared__ πίνακες σε [33][33], εξασφαλίζοντας τη μέγιστη δυνατή ταχύτητα χωρίς memory conflicts.